

Universidade do Minho
Escola de Engenharia

Segurança de Sistemas Informáticos

2025/26

Chat Seguro E2EE

Relatório Técnico — Projeto Final

João Tomás Maciel Sousa - A106900

José Dinis Guimarães Machado - A106926

Pedro Afonso Quesado Pacheco Pereira - A107332

24 de Maio de 2026

1.Introdução	3
2. Arquitetura do Sistema	4
2.1 Visão Geral	4
2.2 Protocolo de Comunicação de Rede	4
2.2.1 Justificação da Ausência de TLS	5
2.3 Gestão de Identidades e Arranque	5
3. Modelo de Segurança e Primitivas Criptográficas	7
3.1 Pressupostos e Modelo de Ameaças	7
3.2 Criptografia Assimétrica — RSA-2048	7
3.2.1 Assinaturas Digitais — RSA-PSS	7
3.2.2 Encapsulamento de Chaves — RSA-OAEP	8
3.3 Criptografia Simétrica — AES-256-GCM	8
3.4 Troca de Chaves Efémera — ECDH X25519	8
3.5 Derivação de Chave de Sessão — HKDF-SHA256	9
4. Fluxos de Comunicação	10
4.1 Autenticação Mútua — Protocolo Challenge-Response	10
4.2 Autoridade de Certificação (PKI Customizada)	10
4.2.1 Estrutura do Certificado	11
4.3 Envio de Mensagem 1-para-1	11
4.3.1 Modo Clássico (Fallback KEM)	11
4.3.2 Modo Forward Secrecy	12
4.4 Protecção Anti-Replay	13
5. Valorizações Implementadas	14
5.1 Mensagens Offline	14
5.1.1 Fluxo de Armazenamento e Entrega	14
5.2 PKI / Entidade de Certificação	14
5.3 Forward Secrecy	15
5.4 Mensagens de Grupo	15
5.4.1 Modelo de Distribuição de Chaves	15
5.4.2 Envio de Mensagens de Grupo	15
5.5 Modo PGP-like (Comunicação Descentralizada)	16
5.5.1 Estrutura do Envelope PGP	16
5.5.2 Garantias de Segurança do Modo PGP	16
6. Garantias de Segurança e Limitações	18
6.1 Garantias Oferecidas	18
6.2 Limitações Identificadas	18
6.2.1 HKDF sem Salt Aleatório	18
6.2.2 AES-GCM sem AAD	18
6.2.3 Ausência de Revogação de Certificados	19
6.2.4 Persistência em Plaintext no Disco do Servidor	19
6.2.5 Sessões Simultâneas por Utilizador	19
7. Conclusão	20

1.Introdução

O presente projeto, desenvolvido no âmbito da unidade curricular de Segurança de Sistemas Informáticos (SSI) da Universidade do Minho, consiste na conceção e implementação de um sistema de chat seguro com cifragem ponta-a-ponta (End-to-End Encryption, E2EE) em Python, recorrendo exclusivamente à biblioteca *cryptography* para todas as operações criptográficas.

O sistema foi construído com base em dois pressupostos centrais: o servidor é honesto mas curioso — cumpre corretamente as suas funções de encaminhamento e gestão, mas não deve ter acesso ao conteúdo das mensagens — e um atacante externo pode tentar interceptar e manipular ativamente o tráfego de rede, incluindo ataques de Man-in-the-Middle.

Para além dos requisitos base de confidencialidade, integridade e autenticidade, o projeto inclui cinco valorizações:

Valorização	Descrição
Mensagens Offline	Armazenamento e entrega diferida de mensagens cifradas quando o destinatário está desligado
PKI / CA Customizada	O servidor actua como Autoridade de Certificação self-signed, emitindo certificados digitais associados às identidades dos utilizadores
Forward Secrecy	Troca de chaves ECDH X25519 efémeras por sessão; comunicações passadas ficam protegidas mesmo que a chave RSA seja comprometida
Mensagens de Grupo	Chave AES de grupo distribuída individualmente por RSA-OAEP; o servidor nunca tem acesso à chave de grupo
Modo PGP-like	Comunicação offline através de ficheiros .pgp com cifragem RSA-OAEP, assinatura RSA-PSS e independência total do servidor

2. Arquitetura do Sistema

2.1 Visão Geral

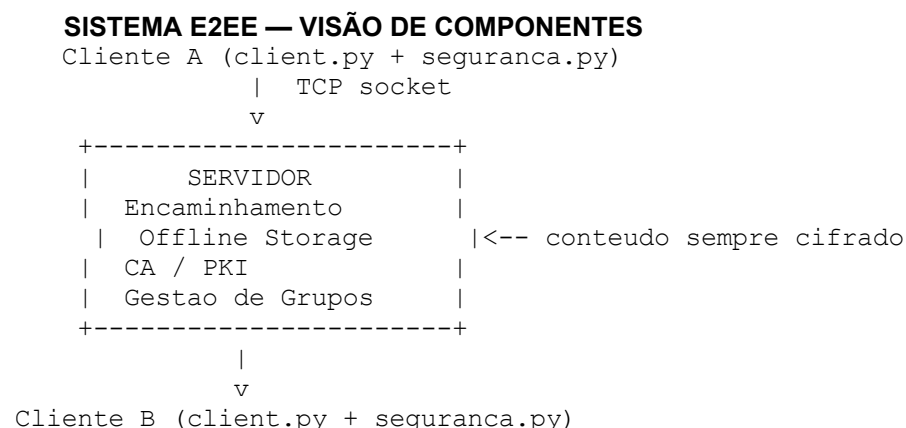
O sistema segue um modelo cliente-servidor assente em sockets TCP. O servidor funciona como ponto central de coordenação — encaminha mensagens, mantém o estado dos utilizadores online, armazena mensagens para destinatários offline e atua como Autoridade de Certificação — mas nunca tem acesso ao conteúdo em claro das mensagens trocadas entre clientes. Cada cliente corresponde a uma instância da aplicação utilizada por um utilizador, que interage com o sistema através de um conjunto de comandos textuais.

O projeto está dividido em cinco ficheiros Python com responsabilidades claramente separadas:

Ficheiro	Responsabilidade
seguranca.py	Biblioteca de todas as primitivas criptográficas (RSA, AES-GCM, ECDH, HKDF, PKI, anti-replay)
server.py	Servidor central: autenticação mútua, CA, encaminhamento de mensagens, armazenamento offline, gestão de grupos
client.py	Cliente E2EE: handshake, envio/receção cifrada, comandos do utilizador, grupos, modo PGP
utils.py	Utilitários de rede: framing TCP com prefixo de comprimento e serialização binária de tuplos
gerar_chaves.py	Utilitário de setup: geração de pares RSA-2048 para os utilizadores pré-definidos

A separação entre `seguranca.py` e o restante código foi uma decisão de design deliberada: toda a lógica criptográfica está isolada num único módulo, o que facilita a auditoria, os testes unitários e a reutilização das primitivas tanto pelo cliente como pelo servidor.

O diagrama seguinte ilustra os componentes do sistema e o fluxo geral de comunicação:



2.2 Protocolo de Comunicação de Rede

A camada de transporte assenta em sockets TCP com um protocolo de framing simples mas robusto, implementado em `utils.py`. Como o TCP é um protocolo orientado a fluxo de bytes — e não a mensagens discretas — é necessário delimitar explicitamente cada mensagem para evitar problemas de fragmentação ou concatenação de pacotes (os chamados short reads).

A solução adoptada utiliza um prefixo de 4 bytes em big-endian que indica o tamanho total do payload que se segue:

4 bytes(tamanho, big-endian)	N bytes (payload da mensagem)
------------------------------	-------------------------------

Para a serialização de pares de valores binários (por exemplo, assinatura + dados cifrados), o módulo `utils.py` disponibiliza o esquema `mkpair/unpair`: os primeiros 2 bytes em little-endian indicam o comprimento do primeiro elemento, seguido dos bytes do primeiro e segundo elementos:

2 bytes(len(x), little-endian)	x (len bytes)	y (bytes restantes)
--------------------------------	---------------	---------------------

Esta representação binária compacta evita o overhead de formatos como JSON para dados que são intrinsecamente binários, como assinaturas digitais e ciphertext.

2.2.1 Justificação da Ausência de TLS

A exclusão de uma camada de transporte TLS estandardizada resulta de duas exigências arquiteturais do projeto:

1. **Autenticação Mútua Mandatória:** O modelo de ameaças exige que ambas as partes (Cliente e Servidor) provem inequivocamente a sua identidade. Em configurações TLS convencionais, apenas o servidor é autenticado, sendo a autenticação do cliente por certificado frequentemente ignorada. O protocolo *challenge-response* implementado na camada de aplicação força esta validação mútua a cada sessão.
2. **Cifragem End-to-End Aplicacional:** Mesmo num túnel TLS perfeitamente seguro, o tráfego seria decifrado ao chegar ao servidor, comprometendo o requisito fundamental de que o servidor não tenha acesso ao conteúdo das mensagens. Ao efetuar a cifragem na camada aplicacional, asseguramos o E2EE independentemente do transporte.

2.3 Gestão de Identidades e Arranque

Cada utilizador possui um par de chaves RSA-2048 como identidade de longa duração. As chaves são geradas pelo utilitário `gerar_chaves.py` e guardadas em ficheiros PEM nos diretórios `chaves_privadas/` e `chaves_publicas/`. A chave privada pode ser protegida com uma password, caso em que é cifrada com `BestAvailableEncryption` da biblioteca `cryptography` (tipicamente AES-256-CBC com PBKDF2).

O servidor gera automaticamente o seu próprio par RSA ao arrancar pela primeira vez, caso não exista ainda. Este par tem um papel duplo: autentica o servidor no handshake de autenticação mútua e é usado como chave de assinatura da CA para emitir certificados. A chave pública do servidor (`servidor_publica.pem`) é o único elemento que precisa de ser distribuído a priori a todos os clientes — é o root of trust de toda a PKI.

A tabela seguinte resume os ficheiros gerados pelo sistema e a sua classificação de segurança:

Ficheiro	Conteúdo	Classificação
<code>chaves_privadas/<id>_privada.pem</code>	Chave privada RSA do utilizador	SECRETA — nunca partilhar
<code>chaves_publicas/<id>_publica.pem</code>	Chave pública RSA do utilizador	Pública — distribuída na PKI

chaves_publicas/servidor_publica.pem	Chave pública da CA (root of trust)	Pública — distribuída a todos
certificados.json	Certificados PKI emitidos (hex)	Pública — gerida pelo servidor
mensagens_offline.json	Mensagens cifradas pendentes (hex)	Cifrada — servidor não decifra
grupos.json	Metadados de grupos (membros, criador)	Metadados sem conteúdo

3. Modelo de Segurança e Primitivas Criptográficas

3.1 Pressupostos e Modelo de Ameaças

Antes de justificar qualquer escolha criptográfica, é essencial definir com precisão o que o sistema pretende proteger e contra quem. O modelo de ameaças adoptado assenta em três pressupostos fundamentais:

Servidor honesto mas curioso (honest-but-curious). O servidor executa corretamente todas as funções de encaminhamento, gestão de utilizadores e armazenamento. Contudo, pode tentar ler o conteúdo das mensagens e dos dados armazenados. Por esta razão, toda a informação sensível é cifrada antes de chegar ao servidor — incluindo as mensagens offline, que ficam armazenadas em formato cifrado sem que o servidor tenha acesso à chave.

Atacante de rede activo (Dolev-Yao). Assume-se que um adversário pode interceptar, modificar, injectar e reenviar mensagens na rede. Este modelo cobre ataques de Man-in-the-Middle, replay attacks e impersonation. O protocolo de autenticação mútua e as assinaturas digitais em cada mensagem foram desenhados especificamente para resistir a este tipo de adversário.

Comprometimento futuro de chaves de longa duração. Um atacante pode vir a descobrir a chave RSA privada de um utilizador no futuro, por exemplo através de um ataque de força bruta ou de uma falha no armazenamento. O mecanismo de Forward Secrecy garante que este comprometimento não permite decifrar comunicações passadas, cujas chaves efémeras já foram destruídas.

A tabela seguinte resume as propriedades de segurança que o sistema garante e os mecanismos responsáveis por cada uma:

Propriedade	Mecanismo	Âmbito
Confidencialidade	AES-256-GCM com chave efémera por mensagem	Mensagens 1-para-1, grupo e PGP
Integridade	AES-GCM (tag de autenticação integrada)	Todas as mensagens cifradas
Autenticidade	RSA-PSS sobre cada mensagem enviada	Mensagens 1-para-1, grupo e PGP
Autenticação mútua	Challenge-response com RSA-PSS + timestamp	Handshake cliente-servidor
Forward Secrecy	ECDH X25519 efémero + HKDF	Sessões 1-para-1 online
Protecção anti-replay	Registo de nonces AES-GCM (janela 5 min)	Todas as mensagens recebidas
Gestão de confiança (PKI)	Certificados JSON assinados pela CA	Identidade de todos os utilizadores

3.2 Criptografia Assimétrica — RSA-2048

O RSA-2048 é utilizado como primitiva de identidade de longa duração, cumprindo dois papéis distintos: o **encapsulamento de chaves simétricas** (KEM) e as **assinaturas digitais**. A escolha de 2048 bits reflete o equilíbrio entre segurança e desempenho para um contexto académico: o NIST considera o RSA-2048 seguro até pelo menos 2030, oferecendo um nível de segurança equivalente a 112 bits simétricos.

3.2.1 Assinaturas Digitais — RSA-PSS

O sistema adota o esquema de assinatura RSA-PSS (*Probabilistic Signature Scheme*), suportado pela função de *hash* SHA-256 e pela função de geração de máscara MGF1. A

opção pelo PSS em detrimento do PKCS#1 v1.5 baseia-se na sua prova formal de segurança no modelo de oráculo aleatório, imune a vulnerabilidades clássicas como o ataque de Bleichenbacher. O recurso a um *salt* de dimensão máxima assegura que assinaturas múltiplas do mesmo documento resultem em criptogramas imprevisíveis.

A implementação do RSA-PSS protege três vetores cruciais:

1. Prova criptográfica de identidade durante o *handshake* de autenticação.
2. Garantia de autoria (não-repúdio) nas mensagens transmitidas nos diversos modos operacionais.
3. Proteção da integridade da chave efémera no acordo ECDH contra manipulações efetuadas por um atacante ativo.

3.2.2 Encapsulamento de Chaves — RSA-OAEP

Para salvaguardar o trânsito de chaves simétricas (por exemplo, na distribuição de chaves AES de grupo), utilizou-se o esquema RSA-OAEP (*Optimal Asymmetric Encryption Padding*) com MGF1-SHA256. O OAEP atua como uma barreira arquitetural contra ataques baseados em texto cifrado escolhido (CCA) e é empregue estritamente na cifragem de chaves estruturadas de 32 *bytes*, descartando assim a prática desaconselhada de cifrar blocos arbitrários e extensos com o algoritmo RSA de forma direta.

3.3 Criptografia Simétrica — AES-256-GCM

Para a cifragem do conteúdo das mensagens, o sistema usa **AES-256-GCM**. O modo GCM é um modo de cifragem autenticada (AEAD — Authenticated Encryption with Associated Data) que combina confidencialidade e integridade numa única operação, sem necessidade de primitivas separadas.

O GCM funciona combinando dois mecanismos: o modo CTR (Counter Mode) para a cifragem do plaintext, e o GHASH para o cálculo de uma tag de autenticação de 128 bits. A tag é gerada sobre o ciphertext e eventuais dados adicionais (AAD), garantindo que qualquer adulteração do ciphertext — mesmo de um único bit — é detectada na decifração.

Cada mensagem usa um **nonce aleatório de 12 bytes** gerado com `os.urandom(12)`, o que é o tamanho recomendado pelo NIST SP 800-38D para GCM. A aleatoriedade do nonce é essencial: a reutilização do mesmo nonce com a mesma chave comprometeria completamente a confidencialidade e a autenticidade. Como cada mensagem usa uma chave efémera diferente (no modo clássico) ou um nonce fresco (no modo FS), a probabilidade de colisão é negligenciável.

3.4 Troca de Chaves Efémera — ECDH X25519

Para o mecanismo de Forward Secrecy, o sistema usa **ECDH X25519** (Elliptic-curve Diffie-Hellman com a curva Curve25519). A X25519 foi escolhida em detrimento de alternativas como o DH clássico por três razões principais.

Segurança. A Curve25519 foi desenhada por Daniel J. Bernstein especificamente para resistir a implementações vulneráveis e a ataques de canal lateral. Ao contrário das curvas NIST, cujos parâmetros foram escolhidos por critérios não totalmente transparentes, a Curve25519 tem parâmetros com justificação pública e verificável.

Desempenho. As operações X25519 são significativamente mais rápidas que a RSA-2048 para o mesmo nível de segurança (aproximadamente 128 bits), tornando o handshake efémero praticamente imperceptível para o utilizador.

Implementação segura por defeito. A X25519 usa aritmética de Montgomery em tempo constante, eliminando vulnerabilidades de timing attack que afetam implementações naive de ECDH sobre curvas de Weierstrass.

As chaves efêmeras são serializadas em formato raw de 32 bytes para transmissão pela rede. Após o cálculo do segredo partilhado, a chave privada efêmera é imediatamente destruída, garantindo que não pode ser recuperada mesmo que o processo seja inspeccionado posteriormente.

3.5 Derivação de Chave de Sessão — HKDF-SHA256

O output bruto de uma troca ECDH não deve ser usado diretamente como chave criptográfica, pois pode não ter a distribuição uniforme necessária para AES. O sistema aplica **HKDF-SHA256** (HMAC-based Key Derivation Function, RFC 5869) para derivar a chave de sessão a partir do segredo ECDH.

O HKDF funciona em duas fases: *extract* (comprime o material de entrada em pseudorandom key usando HMAC-SHA256) e *expand* (expande para o comprimento desejado com um parâmetro de contexto). No sistema, é usada a versão simplificada com os seguintes parâmetros:

Parâmetro	Valor	Justificação
Algoritmo	SHA-256	Função de hash segura e amplamente auditada
length	32 bytes (256 bits)	Tamanho exacto de uma chave AES-256
info	"chat-e2ee-sessao-v1"	Binding de contexto; isola esta derivação de outros usos do mesmo segredo
salt	None (não utilizado)	Limitação identificada — salt aleatório aumentaria a segurança

O parâmetro `info` serve como contexto de derivação: garante que o mesmo segredo ECDH, se reutilizado noutro protocolo, produziria uma chave diferente. A ausência de `salt` é uma limitação menor — um salt aleatório fornecido por ambas as partes aumentaria a entropia da derivação, mas o HKDF sem salt continua a ser seguro desde que o segredo ECDH seja suficientemente entrópico, o que a X25519 garante.

4. Fluxos de Comunicação

4.1 Autenticação Mútua — Protocolo Challenge-Response

Quando um cliente se liga ao servidor, é obrigatório executar um protocolo de autenticação mútua antes de qualquer outra operação. O protocolo garante que o cliente prova ao servidor que possui a chave privada correspondente à sua identidade, e que o servidor prova simetricamente ao cliente a sua própria identidade — eliminando a possibilidade de um atacante se fazer passar por qualquer das partes.

O mecanismo base é um **challenge-response com assinatura RSA-PSS**: quem desafia gera um valor aleatório de 32 bytes (dos quais os últimos 4 são um timestamp Unix), e o desafiado tem de assinar esse valor com a sua chave privada. A assinatura só pode ser produzida por quem possui a chave privada, e o timestamp embutido impede que uma assinatura legítima seja reutilizada depois de 30 segundos (replay attack no handshake).

O protocolo é composto por 7 passos e inclui simultaneamente o registo PKI do cliente:

#	Direcção	Mensagem	Descrição
1	Servidor → Cliente	DESAFIO: nonce(28B) ts(4B)	Servidor gera desafio com timestamp embutido
2	Cliente → Servidor	ID_AUTH: id RSA-PSS(desafio)	Cliente prova posse da chave privada
3	Cliente → Servidor	CHAVE_PUB: PEM da chave pública	Cliente envia chave pública para emissão de certificado
4	Servidor → Cliente	CERT: certificado JSON assinado pela CA	CA emite e envia certificado com validade de 1 ano
5	Cliente → Servidor	DESAFIO_CLI: nonce(28B) ts(4B)	Cliente desafia o servidor (autenticação mútua)
6	Servidor → Cliente	RESP_SERVIDOR: RSA-PSS(desafio_cli)	Servidor prova posse da sua chave privada
7	Cliente → Servidor	AUTH_OK	Cliente confirma autenticação — ligação segura estabelecida

É de salientar que os passos 3 e 4 estão integrados no fluxo de autenticação, evitando a sobrecarga de ligações adicionais (*round-trips*) apenas para o provisionamento de certificados. Após o passo 7, o cliente dispõe de um certificado PKI válido, isentando o servidor de recorrer a ficheiros locais para validar futuras ligações deste utilizador.

4.2 Autoridade de Certificação (PKI Customizada)

O servidor atua como Autoridade de Certificação (CA) self-signed, sendo o único emissor de certificados no sistema. A sua chave pública é o root of trust — o único elemento que os clientes precisam de ter a priori para verificar qualquer certificado.

4.2.1 Estrutura do Certificado

Cada certificado é uma estrutura JSON serializada de forma determinística (`sort_keys=True`, necessário para que a assinatura seja verificável), com os seguintes campos:

Campo	Tipo	Descrição
id	string	Identificador único do utilizador
chave_pub	string (PEM)	Chave pública RSA do utilizador em formato PEM
emitido_em	int (Unix timestamp)	Data de emissão — protege contra certificados com data futura suspeita
valido_ate	int (Unix timestamp)	Data de expiração — validade de 365 dias a partir da emissão
versao	int (sempre 1)	Versão do formato do certificado, para compatibilidade futura

Para a transmissão na rede, o certificado é serializado num bloco binário contíguo: [Tamanho do JSON em 4 bytes (Big-Endian)] + [Corpo JSON] + [Assinatura RSA-PSS da CA]. O cliente, ao inspecionar o certificado, aplica validações estritas por ordem crescente de complexidade computacional: verificação temporal (*timestamps*), seguida da validação da assinatura criptográfica da CA. A grande vantagem arquitetural dessa abordagem face à partilha manual de chaves é a **escalabilidade intrínseca**: um cliente descobre e valida novos utilizadores dinamicamente através do comando `CMD: CERT:<id>`.

4.3 Envio de Mensagem 1-para-1

O sistema suporta dois modos de envio de mensagens entre utilizadores: o modo clássico (fallback, baseado em RSA-OAEP) e o modo com Forward Secrecy (preferido, baseado em ECDH X25519). O modo FS é ativado automaticamente quando o utilizador executa `/conectar`, iniciando um handshake ECDH efémero.

4.3.1 Modo Clássico (Fallback KEM)

O modo clássico é utilizado quando o handshake ECDH ainda não completou — por exemplo, nas primeiras mensagens enviadas imediatamente após `/conectar`, ou quando o destinatário ainda não respondeu ao `FS_HELLO`. A estrutura do envelope é a seguinte:

ENVELOPE MODO CLÁSSICO

```
MSG: mkipair( RSA-PSS(envelope_interno) , envelope_interno )
      |
      |
envelope_interno = mkipair( RSA-OAEP(chave_AES) , AES-GCM(msg) )
                        |           |
                        |           |
                chave RSA publica  nonce 12B + ciphertext
                do destinatario    + tag GCM 16B
```

O receptor: (1) separa assinatura e `envelope_interno` com `unpair`; (2) verifica a assinatura RSA-PSS com a chave pública do remetente — rejeita se inválida; (3) extrai e verifica o nonce AES-GCM contra o registo anti-replay — rejeita se já visto; (4) decifra a chave AES com a sua chave privada RSA-OAEP; (5) decifra a mensagem com AES-GCM. Cada mensagem usa uma chave AES gerada aleatoriamente (`os.urandom(32)`), pelo que não existe reutilização de chaves simétricas entre mensagens.

4.3.2 Modo Forward Secrecy

Quando o utilizador executa `/conectar <nome>`, o cliente inicia automaticamente um handshake ECDH X25519 efémero com o destinatário, usando o servidor como relay. O objectivo é estabelecer uma chave de sessão partilhada que não está ligada às chaves RSA de longa duração — garantindo que a descoberta futura da chave RSA não compromete as mensagens já enviadas.

O handshake funciona em qualquer ordem de `/conectar`: se Bob ainda não fez `/conectar` alice, ao receber o FS_HELLO de Alice, o seu cliente responde automaticamente sem intervenção do utilizador. O protocolo é o seguinte:

ENVELOPE MODO CLÁSSICO

```
MSG: mkpair( RSA-PSS(envelope_interno) , envelope_interno )
      |
      envelope_interno = mkpair( RSA-OAEP(chave_AES) , AES-GCM(msg) )
                        |                               |
                        chave RSA publica             nonce 12B + ciphertext
                        do destinatário                 + tag GCM 16B
```

#	Direcção	Mensagem	Operação local
1	Alice → Bob	FS_HELLO: mkpair(pub_ecdh_A, RSA-PSS(pub_ecdh_A id_A id_B))	Alice gera par X25519 efémero; assina a chave com RSA
2	Bob → Alice	FS_HELLO: mkpair(pub_ecdh_B, RSA-PSS(pub_ecdh_B id_B id_A))	Bob verifica assinatura, gera par X25519, calcula segredo, destrói priv_B
3		(sem mensagem adicional)	Alice recebe FS_HELLO de Bob, verifica assinatura, calcula segredo, destrói priv_A

Após o passo 3, ambos têm o mesmo segredo ECDH e derivam a mesma chave de sessão AES-256 via HKDF-SHA256. As chaves privadas efémeras são destruídas (`priv = None`) imediatamente após o cálculo — este é o momento em que a propriedade de Forward Secrecy fica garantida. A assinatura RSA-PSS sobre `pub_ecdh || id_remetente || id_destinatario` previne que um atacante MITM substitua a chave ECDH de uma das partes, pois a substituição invalidaria a verificação da assinatura do outro lado.

Uma vez estabelecida a chave de sessão, as mensagens passam a usar o modo FS, com um envelope significativamente mais simples:

ENVELOPE MODO FORWARD SECRECY

```
MSG_FS: mkpair( RSA-PSS(ciphertext) , AES-GCM(msg, chave_sessao_ECDH) )
              |
              chave derivada via HKDF do segredo X25519
              (chave privada efemera ja destruida)
```

A comparação com o modo clássico é imediata: desaparece a necessidade de cifrar uma chave AES com RSA-OAEP por mensagem, pois a chave de sessão é partilhada para toda a conversa. O custo computacional por mensagem reduz-se a uma cifragem AES-GCM e uma assinatura RSA-PSS, sendo o RSA-OAEP usado apenas uma vez no handshake inicial.

4.4 Protecção Anti-Replay

Uma vulnerabilidade clássica em sistemas de mensagens cifradas é o replay attack: um atacante que intercepte um pacote cifrado legítimo pode reenviar esse mesmo pacote mais tarde, sem necessidade de o decifrar. A mensagem seria aceite como legítima pelo receptor, causando duplicação de mensagens ou outras consequências indesejadas.

O sistema implementa protecção anti-replay através da classe `AntiReplay`, que mantém um registo dos nonces AES-GCM de todas as mensagens já processadas. Como cada mensagem usa um nonce gerado aleatoriamente com `os.urandom(12)`, a probabilidade de colisão genuína é negligenciável (2^{-96}), pelo que qualquer nonce repetido indica um replay attack.

O registo tem uma janela de 5 minutos: nonces com mais de 300 segundos são removidos automaticamente em cada operação de verificação, evitando crescimento ilimitado de memória. A protecção anti-replay é aplicada de forma consistente em todos os tipos de mensagem: modo clássico, modo FS e mensagens de grupo.

5. Valorizações Implementadas

Para além dos requisitos base do enunciado, o sistema implementa cinco valorizações que reforçam tanto a segurança como a funcionalidade da aplicação. Esta secção descreve cada uma, enquadrando-a no modelo de ameaças e detalhando as decisões de design específicas.

5.1 Mensagens Offline

Um sistema de chat em tempo real tem uma limitação funcional óbvia: se o destinatário não estiver ligado, a mensagem perde-se. A valorização de mensagens offline resolve este problema mantendo o modelo de segurança intacto — o servidor armazena mensagens sem nunca ter acesso ao seu conteúdo.

5.1.1 Fluxo de Armazenamento e Entrega

Quando Alice envia uma mensagem a Bob e o servidor detecta que Bob não está online, o pacote cifrado completo é guardado em `mensagens_offline.json`, codificado em hexadecimal. O servidor não tenta decifrar, inspecionar ou modificar o pacote — armazena exactamente o que recebeu. Quando Bob se liga e completa o handshake de autenticação, o servidor entrega os pacotes pendentes em ordem de chegada e limpa o registo.

```

                                FLUXO DE MENSAGEM OFFLINE
Alice                               Servidor                               Bob (offline)
|-- DEST:bob | ENV -->|                                     |
|                                     | Bob nao esta online |
|                                     |-- guarda ENV em JSON ----->|
|                                     | (hex, sem decifrar) |
|                                     (Bob liga-se mais tarde)
|                                     |<-- handshake autent. -----|
|                                     |-- ENV (pendente) ----->|
|                                     |-- limpa registo offline -->|
```

A garantia de segurança é directa: o servidor é um **relay cego**. O conteúdo armazenado é exactamente o mesmo envelope cifrado que Bob receberia se estivesse online — cifragem E2EE, assinatura RSA-PSS e nonce AES-GCM incluídos. A protecção anti-replay continua a aplicar-se na entrega, pelo que um atacante que modifique o ficheiro JSON no servidor não consegue injectar mensagens válidas.

5.2 PKI / Entidade de Certificação

O enunciado identifica como modelo base a pré-partilha manual de identidades entre os participantes — ou seja, cada cliente teria de ter previamente os ficheiros `.pem` de todos os outros. Esta abordagem não escala e é impraticável num cenário real.

A PKI implementada resolve este problema: o servidor atua como **CA self-signed**, emitindo um certificado assinado com RSA-PSS a cada utilizador no momento do registo. A partir daí, qualquer cliente pode obter a chave pública verificada de outro utilizador pedindo o seu certificado ao servidor (`CMD:CERT:<id>`) e verificando a assinatura da CA localmente. O servidor não precisa de ser confiável para este processo — a verificação é feita pelo cliente com a chave pública da CA, que já tem.

A estrutura do certificado e o protocolo de emissão estão detalhados na secção 4.2. O impacto prático desta valorização é que o sistema passa de um modelo de confiança

pré-estabelecida (Web of Trust manual) para um modelo PKI centralizado, com o servidor como root of trust único e verificável.

5.3 Forward Secrecy

O modelo clássico de cifragem assimétrica tem uma vulnerabilidade estrutural conhecida onde um adversário pode gravar tráfego cifrado hoje e aguardar até conseguir comprometer a chave RSA privada de um utilizador — seja por avanços computacionais, por uma falha de segurança ou por outros meios. Com a chave RSA em mão, todas as comunicações passadas ficam expostas.

A Forward Secrecy elimina esta vulnerabilidade: como as chaves de sessão são derivadas de pares ECDH X25519 efêmeros que são destruídos após uso, **não existe nenhuma chave que permita decifrar comunicações passadas**, mesmo que a chave RSA seja posteriormente comprometida. Cada sessão /conectar gera um novo par efêmero, pelo que sessões diferentes têm chaves de sessão independentes.

O protocolo do handshake ECDH e a estrutura do envelope FS estão detalhados na secção 4.3.2. É importante notar que a Forward Secrecy se aplica apenas às sessões online em tempo real — as mensagens offline usam o modo clássico com RSA-OAEP, pois o handshake ECDH requer interacção em tempo real entre os dois clientes.

5.4 Mensagens de Grupo

O suporte a grupos de chat introduz um desafio criptográfico adicional: como cifrar mensagens de forma que todos os membros do grupo as possam decifrar, sem que o servidor tenha acesso à chave partilhada, e sem que um membro não autorizado possa participar?

5.4.1 Modelo de Distribuição de Chaves

O modelo adoptado é baseado em KEM individual por membro: o criador do grupo gera uma chave AES-256 aleatória para o grupo e distribui-a a cada membro individualmente, cifrando uma cópia com a chave pública RSA-OAEP de cada um. O servidor gere a lista de membros mas nunca tem acesso à chave de grupo — recebe e encaminha envelopes opacos.

DISTRIBUIÇÃO DA CHAVE DE GRUPO (KEM INDIVIDUAL)

```
Criador (Alice)
1. chave_grupo = AES-256 aleatorio
2. Para cada membro (Bob, Carlos, ...):
    env_membro = RSA-OAEP(pub_membro, chave_grupo)
    sig         = RSA-PSS(priv_alice, env_membro)
    envia DEST:membro | GRUPO_CHAVE:nome:env_membro|sig
Cada membro ao receber:
1. Verifica RSA-PSS de Alice (autenticidade)
2. Decifra RSA-OAEP com a sua chave privada -> chave_grupo
```

5.4.2 Envio de Mensagens de Grupo

Com a chave de grupo distribuída, o envio de mensagens é simples: o remetente cifra com AES-256-GCM usando a chave partilhada, assina o ciphertext com RSA-PSS, e envia o envelope para o servidor com o prefixo GRUPO_MSG:<nome>. O servidor encaminha para todos os membros online e armazena offline para os que estão desligados — em ambos os casos sem acesso ao conteúdo.

Cada receptor verifica a assinatura RSA-PSS do remetente antes de decifrar, garantindo que apenas membros legítimos com a chave privada correspondente podem enviar mensagens válidas para o grupo. A protecção anti-replay aplica-se igualmente às mensagens de grupo.

5.5 Modo PGP-like (Comunicação Descentralizada)

O modo PGP-like permite a troca de mensagens cifradas **completamente independente do servidor** — incluindo quando o servidor está desligado. Inspirado no modelo de cifragem de ficheiros do PGP (Pretty Good Privacy), este modo exporta mensagens para ficheiros .pgp que podem ser transmitidos por qualquer canal fora de banda: email, USB, Bluetooth, etc.

5.5.1 Estrutura do Envelope PGP

Cada ficheiro .pgp é um JSON com os seguintes campos:

Garantia	Mecanismo no modelo de grupo
Confidencialidade	AES-256-GCM com chave de grupo; servidor nunca tem a chave
Autenticidade do remetente	RSA-PSS sobre o ciphertext por cada mensagem enviada
Controlo de acesso	Só quem recebeu a chave via RSA-OAEP pode decifrar mensagens do grupo
Distribuição segura da chave	KEM individual RSA-OAEP por membro + assinatura RSA-PSS do criador
Anti-replay	Registo de nonces AES-GCM, janela de 5 minutos

5.5.2 Garantias de Segurança do Modo PGP

O envelope PGP oferece o mesmo conjunto de garantias que o modo online, adaptado ao contexto assíncrono:

Campo	Tipo	Conteúdo
versao	int (1)	Versão do formato, para compatibilidade futura
remetente	string	ID do utilizador que criou o ficheiro
destinatario	string	ID do destinatário — verificado na importação
timestamp	int (Unix)	Data e hora de criação do envelope
chave_aes_cifrada	hex string	Chave AES efémera cifrada com RSA-OAEP da chave pública do destinatário
mensagem_cifrada	hex string	Mensagem cifrada com AES-256-GCM (nonce ciphertext tag)
assinatura	hex string	RSA-PSS sobre chave_aes_cifrada mensagem_cifrada — garante integridade de tudo

A assinatura RSA-PSS cobre **tanto a chave AES cifrada como o ciphertext** (chave_aes_cifrada || mensagem_cifrada). Esta escolha é deliberada: assinar apenas o ciphertext deixaria em aberto a possibilidade de um atacante substituir a chave AES cifrada por outra (key substitution attack). Ao assinar os dois campos em conjunto, qualquer modificação — seja no conteúdo seja na chave encapsulada — invalida a assinatura.

Na importação, o cliente executa as verificações por ordem crescente de custo: (1) parse do JSON e validação da estrutura; (2) verificação de que o campo destinatário corresponde ao utilizador local; (3) verificação da assinatura RSA-PSS; (4) decifração RSA-OAEP da chave AES; (5) decifração AES-GCM. Qualquer falha numa etapa resulta numa exceção com mensagem explicativa, sem expor informação sobre as etapas seguintes.

6. Garantias de Segurança e Limitações

6.1 Garantias Oferecidas

A tabela seguinte consolida todas as propriedades de segurança garantidas pelo sistema, indicando o mecanismo responsável, o componente do código onde está implementado e o âmbito de aplicação:

Propriedade	Mecanismo	Componente	Âmbito
Confidencialidade	AES-256-GCM	cifrar_mensagem()	Todas as mensagens
Integridade	Tag GCM 128-bit	AESGCM.decrypt()	Todas as mensagens
Autenticidade	RSA-PSS por mensagem	assinar_dados()	Msgs 1-para-1, grupo, PGP
Autenticação mútua	Challenge-response RSA-PSS + timestamp	handshake_autenticacao()	Arranque de sessão
Forward Secrecy	ECDH X25519 efêmero + HKDF	gerar_par_ecdh_efemero()	Sessões 1-para-1 online
Anti-replay	Registo de nonces GCM, janela 5 min	AntiReplay.verificar_e_registar()	Todas as mensagens
Gestão de identidades (PKI)	Certificados JSON assinados por CA	criar_certificado() / verificar_certificado()	Todos os utilizadores
Não-repúdio	RSA-PSS com chave privada do remetente	assinar_dados()	Modo PGP-like
Controlo de acesso a grupos	KEM individual RSA-OAEP por membro	cifrar_chave_aes_com_rsa()	Mensagens de grupo
Servidor cego (E2EE)	Cifragem aplicacional pré-transmissão	Toda a camada seguranca.py	Todo o sistema

6.2 Limitações Identificadas

A análise honesta de um sistema de segurança exige identificar não apenas o que funciona, mas também o que poderia ser melhorado. As limitações seguintes foram identificadas durante o desenvolvimento e são inerentes às decisões de design tomadas.

6.2.1 HKDF sem Salt Aleatório

A derivação de chave de sessão usa HKDF-SHA256 com `salt=None`, o que faz o HKDF usar internamente um salt de zeros. Um salt aleatório — por exemplo, gerado por cada parte e trocado durante o handshake ECDH — aumentaria a entropia da derivação e tornaria a chave de sessão independente mesmo que o segredo ECDH tivesse alguma correlação entre sessões. Na prática, dado que a X25519 gera segredos com entropia elevada, o impacto desta limitação é reduzido, mas não nulo do ponto de vista formal.

6.2.2 AES-GCM sem AAD

As operações AES-GCM são realizadas sem **Additional Associated Data (AAD)**. O AAD permite autenticar dados em claro — como o identificador do remetente, do destinatário ou o tipo de mensagem — vinculando-os criptograficamente ao ciphertext. A ausência de AAD

significa que um atacante com acesso ao canal poderia, em teoria, re-encaminhar um envelope cifrado legítimo alterando apenas o cabeçalho de routing (DEST:<nome>) sem invalidar a autenticação GCM. A assinatura RSA-PSS sobre o envelope mitiga parcialmente este risco, mas não é um substituto completo para o binding de contexto que o AAD proporciona.

6.2.3 Ausência de Revogação de Certificados

A PKI implementada não inclui nenhum mecanismo de revogação de certificados — equivalente a uma Certificate Revocation List (CRL) ou a OCSP. Se a chave privada de um utilizador for comprometida, o seu certificado continua válido até expirar (até 1 ano). Nesse cenário, um atacante com a chave privada comprometida poderia autenticar-se no servidor e ser aceite como o utilizador legítimo. A mitigação existente é a janela de 30 segundos no challenge-response, que impede a reutilização de assinaturas capturadas, mas não protege contra um atacante com posse activa da chave privada.

6.2.4 Persistência em Plaintext no Disco do Servidor

Os ficheiros `mensagens_offline.json`, `certificados.json` e `grupos.json` são armazenados em plaintext no disco do servidor. Embora o conteúdo das mensagens esteja cifrado (o servidor é um relay cego), os metadados dos grupos e os certificados ficam expostos a quem tenha acesso ao sistema de ficheiros do servidor. Numa implementação de produção, estes ficheiros deveriam ser armazenados numa base de dados com controlo de acesso adequado ou com cifragem ao nível do sistema de ficheiros.

6.2.5 Sessões Simultâneas por Utilizador

O sistema não suporta múltiplas sessões simultâneas para o mesmo utilizador: se Alice se ligar com um segundo cliente enquanto o primeiro ainda está activo, o servidor fecha a sessão mais antiga. Isto pode ser inconveniente em cenários de uso real (múltiplos dispositivos), mas simplifica o modelo de gestão de estado e elimina ambiguidades na entrega de mensagens offline.

7. Conclusão

O sistema de *chat* seguro E2EE desenvolvido neste projeto cumpre integralmente os requisitos definidos, destacando-se pela implementação eficaz de cinco valorizações técnicas avançadas. A arquitetura concebida assenta numa composição rigorosa de primitivas criptográficas modernas (AES-256-GCM, RSA-PSS, RSA-OAEP e ECDH X25519) que garantem as propriedades fundamentais de confidencialidade, integridade, autenticidade e *Forward Secrecy* no espectro comunicacional.

A decisão estrutural de aplicar a segurança na camada aplicacional, em detrimento do padrão TLS, provou-se essencial neste contexto. Esta abordagem viabilizou a imposição da autenticação mútua via *challenge-response* e garantiu o E2EE face a um servidor *honest-but-curious*. Em simultâneo, a integração de uma PKI centralizada resolveu os desafios clássicos de escalabilidade associados à pré-partilha manual de chaves públicas.

As limitações identificadas — como a ausência de AAD e a não revogação de certificados — não infirmam as métricas de segurança concebidas para o modelo de ameaças definido, constituindo o ponto de partida natural para a evolução da plataforma (e.g., adoção do *Double Ratchet*).

Em suma, do ponto de vista pedagógico, o projeto demonstrou na prática que a inviolabilidade de um sistema seguro resulta da sua composição simbiótica e da adequação a um modelo de ameaças concreto, não dependendo exclusivamente do emprego isolado de primitivas criptográficas.